

Phase 3 — L'agent PyTutor

Quatre concepts essentiels : `@tool`, `create_agent`, boucle ReAct, mémoire conversationnelle

Cours PyTutor2

Résumé

Cette phase 3 construit un *agent* qui orchestre les briques des phases précédentes via une boucle *Réfléchir-Agir-Observer* (ReAct). On y aborde : le décorateur `@tool` et la docstring comme contrat avec le LLM, la construction de l'agent avec `create_agent` (LangChain 1.x), l'observation de la boucle ReAct sur trois démos contrastées, et la gestion de la mémoire conversationnelle pour un chat multi-tour. Chaque exercice suit la structure habituelle (énoncé + objectifs en boîte, solution détaillée, vérification LangSmith, récapitulatif).

Table des matières

1	Définir des <i>tools</i> avec <code>@tool</code>	2
1.1	Solution	2
1.1.1	Architecture	2
1.1.2	Le code complet	2
1.1.3	Explications pas à pas	5
1.2	Vérification dans LangSmith	7
2	Construire l'agent avec <code>create_agent</code>	8
2.1	Solution	8
2.1.1	Architecture	8
2.1.2	Le code complet	8
2.1.3	Explications pas à pas	9
2.2	Vérification dans LangSmith	11
3	Observer la boucle Réfléchir-Agir-Observer (ReAct)	12
3.1	Solution	12
3.1.1	Architecture	12
3.1.2	Le code complet	12
3.1.3	Explications pas à pas	14
3.2	Vérification dans LangSmith	15
4	Mémoire conversationnelle multi-tours	16
4.1	Solution	16
4.1.1	Architecture	16
4.1.2	Le code complet	17
4.1.3	Explications pas à pas	17
4.2	Vérification dans LangSmith	19

1 Définir des *tools* avec @tool

Exercice 1. Quatre outils pour l'agent PyTutor

Implémente quatre *tools* que l'agent PyTutor pourra appeler : `search_python_docs` (recherche dans la doc Python via le RAG de la Phase 2), `run_python_code` (exécute du code en subprocess), `generate_exercise` (utilise la chaîne d'explication de la Phase 1), et `check_pep8` (lance `pycodestyle` sur du code).

Chaque tool doit être décoré avec `@tool` de `langchain_core.tools`, et sa **docstring** doit décrire précisément à quoi il sert et *quand* l'appeler — c'est ce que le modèle va lire pour décider.

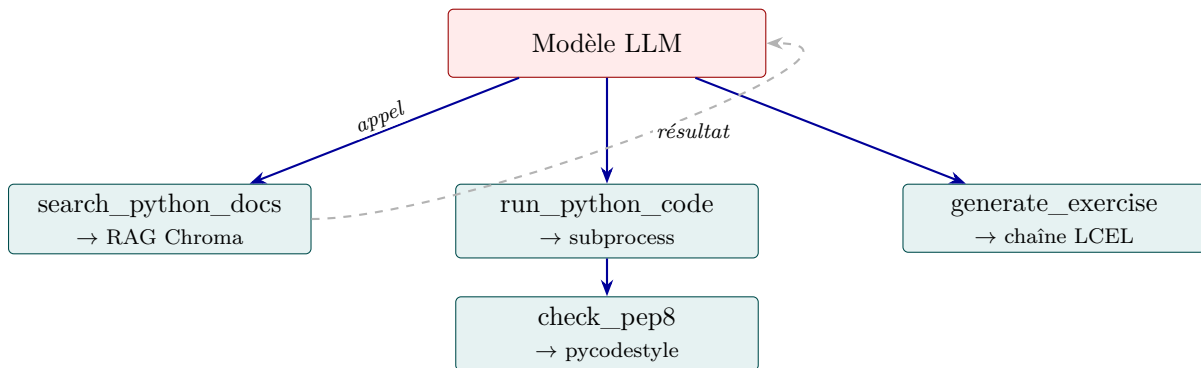
Contrainte : gérer proprement les cas d'erreur (timeout du subprocess, module absent, pas de résultat dans le RAG, valeur invalide en argument) avec des messages qui parlent au LLM.

Objectifs pédagogiques

- Maîtriser le décorateur `@tool` : il transforme une fonction Python en *tool* callable par un LLM.
- Comprendre que la **docstring** est envoyée au modèle comme description du tool — donc c'est du *prompt engineering* appliqué à du code.
- Apprendre les patterns d'implémentation propres : lazy init, `tempfile` avec nettoyage, `timeout`, fallbacks sur erreur.
- Réutiliser les briques des Phases 1 et 2 à l'intérieur d'un tool.

1.1 Solution

1.1.1 Architecture



Le LLM voit 4 **tools déclarés** dans son prompt système (généré par `create_agent`). À chaque tour, il décide d'en appeler un (ou plusieurs) en lisant leur docstring. Le résultat de chaque appel retourne sous forme de `ToolMessage` dans l'historique, et le LLM *réfléchit* à nouveau.

1.1.2 Le code complet

```
1 import subprocess
2 import sys
3 import tempfile
4 from pathlib import Path
5
6 from langchain_core.tools import tool
7
8 from phase1_explain import explain_chain_for_level
9 from phase2_rag import build_or_load_vectorstore
```

```

11
12 # Cache lazy : on charge le retriever une seule fois, à la première utilisation
13 _retriever = None
14
15 def _get_retriever():
16     global _retriever
17     if _retriever is None:
18         vs = build_or_load_vectorstore()
19         _retriever = vs.as_retriever(search_kwargs={"k": 3})
20     return _retriever
21
22
23 @tool
24 def search_python_docs(query: str) -> str:
25     """Recherche dans la documentation Python officielle.
26
27     À utiliser pour toute question FACTUELLE sur Python : syntaxe d'une
28     construction, comportement standard d'une fonction built-in, vocabulaire
29     technique. PAS pour générer du code ou évaluer du code de l'élève.
30
31     Args:
32     query: La question ou les mots-clés à rechercher dans la documentation.
33     """
34     docs = _get_retriever().invoke(query)
35     if not docs:
36         return "Aucun résultat dans la doc Python pour cette recherche."
37     return "\n\n---\n\n".join(
38         f"Source: {d.metadata.get('source', '?')}\n{d.page_content}"
39         for d in docs
40     )
41
42
43 @tool
44 def run_python_code(code: str) -> str:
45     """Exécute du code Python dans un sous-processus et renvoie stdout + stderr.
46
47     À utiliser quand :
48     - l'élève montre un code qui ne marche pas (pour voir la vraie erreur)
49     - tu veux vérifier qu'un exemple que tu vas donner fonctionne réellement
50     - tu veux mesurer la sortie d'un bout de code pour pouvoir l'expliquer
51
52     Limites : timeout 5 secondes, environnement Python standard, pas de pip.
53
54     Args:
55     code: Le code Python à exécuter, complet et autonome.
56     """
57     with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
58         f.write(code)
59         path = f.name
60     try:
61         result = subprocess.run(
62             [sys.executable, path],
63             capture_output=True, text=True, timeout=5,
64         )
65         parts = []
66         if result.stdout:
67             parts.append(f"STDOUT:\n{result.stdout}")
68         if result.stderr:
69             parts.append(f"STDERR:\n{result.stderr}")
70         parts.append(f"Exit code: {result.returncode}")

```

```

71     return "\n\n".join(parts)
72 except subprocess.TimeoutExpired:
73     return (
74         "ERREUR: timeout après 5 secondes. Le code boucle probablement "
75         "à l'infini (boucle while, récursion sans condition d'arrêt, etc.)"
76     )
77 finally:
78     Path(path).unlink(missing_ok=True)
79
80
81 @tool
82 def generate_exercise(topic: str, difficulty: str = "intermédiaire") -> str:
83     """Génère un exercice Python sur un concept avec ses questions de compréhension.
84
85     À utiliser quand l'élève demande explicitement de la pratique, des
86     exercices, ou pour vérifier sa compréhension d'un concept.
87
88     Args:
89         topic: Le concept Python à pratiquer (ex: "décorateurs", "list comprehensions").
90         difficulty: Le niveau visé. Valeurs valides : "débutant", "intermédiaire",
91             ↪ "avancé".
92     """
93     if difficulty not in {"débutant", "intermédiaire", "avancé"}:
94         difficulty = "intermédiaire"
95
96     chain = explain_chain_for_level(difficulty)
97     explanation = chain.invoke({"concept": topic})
98
99     questions = "\n\n".join(
100         f" Q{i}. {q.question}\n      -> Réponse attendue : {q.expected_answer}"
101         for i, q in enumerate(explanation.comprehension_questions, 1)
102     )
103     return (
104         f"Exercice sur '{topic}' (niveau {difficulty}) :\n\n"
105         f"Code de référence :\n{explanation.code_example}\n\n"
106         f"Questions :\n{questions}\n\n"
107         f"Piège classique : {explanation.pitfall}"
108     )
109
110 @tool
111 def check_pep8(code: str) -> str:
112     """Vérifie la conformité PEP 8 d'un bout de code Python.
113
114     À utiliser quand l'élève partage du code et veut savoir s'il est
115     propre, ou quand tu donnes un exemple que tu veux valider stylistiquement.
116
117     Args:
118         code: Le code Python à analyser.
119     """
120     with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
121         f.write(code)
122         path = f.name
123     try:
124         result = subprocess.run(
125             [sys.executable, "-m", "pycodestyle", "--max-line-length=100", path],
126             capture_output=True, text=True, timeout=5,
127         )
128         if not result.stdout.strip():
129             return "Aucune violation PEP 8 détectée."

```

```

130     cleaned = result.stdout.replace(path, "<code>")
131     return f"Problèmes PEP 8 détectés :\n{cleaned}"
132 except FileNotFoundError:
133     return (
134         "Le module pycodestyle n'est pas installé. "
135         "Réponds à l'élève en lui suggérant `pip install pycodestyle`."
136     )
137 finally:
138     Path(path).unlink(missing_ok=True)

```

1.1.3 Explications pas à pas

Étape 1 — Le décorateur @tool

@tool de langchain_core.tools transforme une fonction Python en BaseTool consommable par un LLM. Ce qu'il fait sous le capot :

- Lit la **signature** (noms et types des arguments) pour générer un schéma JSON.
- Lit la **docstring** pour produire la *description* du tool.
- Encapsule l'appel pour intercepter exceptions et timeouts (à compléter manuellement pour les cas avancés).

```

1 @tool
2 def my_tool(query: str) -> str:
3     """Description du tool, en français OK."""
4     return "résultat"
5
6 # my_tool est maintenant un BaseTool, pas une fonction normale
7 my_tool.name          # "my_tool"
8 my_tool.description   # "Description du tool, en français OK."
9 my_tool.args_schema  # Pydantic model avec query: str

```

Étape 2 — La docstring est le contrat avec le LLM

La docstring est **envoyée au modèle** comme description du tool. Le modèle décide quand l'appeler en lisant cette description. Donc :

- **À QUOI** sert le tool — première ligne, factuelle.
- **QUAND** l'utiliser — situations explicites.
- **QUAND PAS** l'utiliser — démarquage des autres tools.
- **Args** typés et décrits — pour que le modèle remplisse bien chaque paramètre.

Une docstring vague = un agent qui prend de mauvaises décisions. Pour chaque tool, demande-toi : « si je donne CETTE docstring à un humain, peut-il savoir QUAND l'appeler sans hésitation ? » Si non, précise.

Étape 3 — Les paramètres typés

Les types Python (str, int, list[str], Literal[...]...) sont traduits en JSON Schema dans la signature envoyée au LLM. Le modèle ne pourra pas appeler le tool avec le mauvais type — c'est validé par Pydantic en amont.

Pour generate_exercise, on accepte un difficulty: str au lieu d'un Literal parce qu'on veut un fallback côté Python plutôt qu'un crash si le modèle invente un niveau "expert". C'est un choix : rigide via Literal pour échouer fort, ou souple via str + validation pour dégrader proprement.

Étape 4 — Patterns d'implémentation

Lazy initialisation. Le retriever du RAG (Phase 2) est lourd à charger. On le *cache* dans une variable de module avec `global` :

```
1 _retriever = None
2
3 def _get_retriever():
4     global _retriever
5     if _retriever is None:
6         vs = build_or_load_vectorstore()
7         _retriever = vs.as_retriever(search_kwargs={"k": 3})
8     return _retriever
```

Le retriever ne se charge que si le tool est effectivement appelé, et seulement la première fois.

Tempfile avec nettoyage. `tempfile.NamedTemporaryFile(delete=False)` + bloc `try/finally` qui supprime explicitement :

```
1 with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
2     f.write(code)
3     path = f.name
4 try:
5     # ... utilise path ...
6     pass
7 finally:
8     Path(path).unlink(missing_ok=True)
```

Timeout sur subprocess. Une boucle `while True` dans le code utilisateur bloquerait l'agent indéfiniment. Le `timeout=5` de `subprocess.run` lève `TimeoutExpired` :

```
1 try:
2     result = subprocess.run(..., timeout=5)
3 except subprocess.TimeoutExpired:
4     return "ERREUR: timeout après 5 secondes. Le code boucle..."
```

Messages d'erreur qui parlent au LLM. Quand un tool échoue, le message de retour devient l'observation que voit le modèle. Mauvais : "Error 127". Bon : "Le module `pycodestyle` n'est pas installé. Réponds à l'élève en lui suggérant `pip install pycodestyle`." — le LLM sait alors quoi répondre.

Un tool n'a pas le droit de planter silencieusement, ni de lever une exception non gérée : dans les deux cas, l'agent perd la main et l'utilisateur voit un crash. Toujours renvoyer un message clair en cas d'erreur, même un message d'erreur.

Étape 5 — Réutiliser les briques des phases précédentes

Chaque tool s'appuie sur ce qu'on a construit précédemment :

Tool	Brique réutilisée
search_python_docs	Vector store + retriever (Phase 2).
run_python_code	subprocess standard, pas LangChain.
generate_exercise	Chaîne <code>explain_chain_for_level</code> avec <code>with_structured_output</code> (Phase 1).
check_pep8	pycodestyle en subprocess.

Un tool peut contenir une chaîne LCEL complète, voire un autre RAG. C'est le pattern « tool qui appelle un autre LLM en interne ». Très puissant pour décomposer un système complexe en agents qui se passent des tâches.

1.2 Vérification dans LangSmith

Quand un tool est appelé par l'agent, LangSmith trace :

```

ToolNode (search_python_docs)
  Input : {"query": "Comment fonctionne yield ?"}
  Output : "Source: docs.python.org/.../functional.html..."
ToolNode (run_python_code)
  Input : {"code": "def f():..."}
  Output : "STDOUT: ... | Exit code: 0"
```

Si un tool ne s'appelle jamais alors qu'il devrait : regarde sa docstring. Si un tool s'appelle trop : regarde aussi sa docstring. C'est presque toujours là.

Récapitulatif

Ce qu'il faut retenir :

1. `@tool` transforme une fonction Python en outil consommable par un LLM. Signature et docstring deviennent un contrat envoyé au modèle.
2. La docstring fait du **prompt engineering** sur le code : explicite à **quoi** sert le tool, **quand** l'appeler, et **quand pas**.
3. Patterns essentiels : `lazy init` pour les ressources lourdes, `tempfile` avec nettoyage, `timeout` sur les subprocess, messages d'erreur qui parlent au LLM.
4. Un tool peut contenir une chaîne LCEL, un RAG, ou n'importe quel autre traitement Python — c'est l'orchestration côté agent qui le rend pertinent.

2 Construire l'agent avec `create_agent`

Exercice 2. Assembler les tools dans un agent ReAct

Construis un agent qui combine : le modèle de chat, la liste des 4 tools définis dans l'exercice 1, et un `system_prompt` qui dicte le comportement attendu (français, pédagogique, règles d'usage des tools).

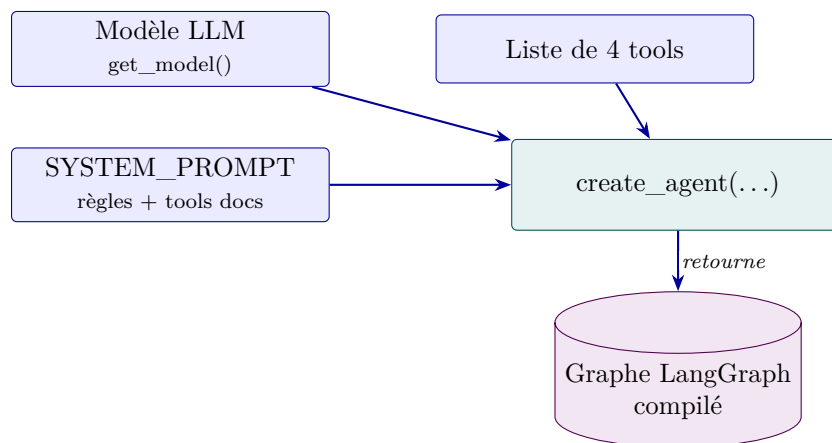
Contrainte : utiliser `create_agent` de `langchain.agents` (LangChain 1.x), qui renvoie un *graphe LangGraph* compilé prêt à `.invoke` ou `.stream`.

Objectifs pédagogiques

- Maîtriser `create_agent(model, tools, system_prompt)` : la voie officielle LangChain 1.x pour les agents ReAct.
- Rédiger un `system_prompt` qui couvre rôle, périmètre, règles de comportement, et choix de langue.
- Comprendre la signature d'invocation : `agent.invoke({"messages": [...]})`.
- Savoir que le retour de `create_agent` est un **graphe LangGraph** (donc tout LangGraph s'applique).

2.1 Solution

2.1.1 Architecture



`create_agent` construit le graphe d'exécution de la boucle ReAct : un nœud LLM qui décide, un nœud ToolNode qui exécute, et des arêtes conditionnelles qui les enchaînent jusqu'à la réponse finale.

2.1.2 Le code complet

```
1 from langchain.agents import create_agent
2 from config import get_model
3
4
5 TOOLS = [search_python_docs, run_python_code, generate_exercise, check_pep8]
6
7 SYSTEM_PROMPT = """Tu es PyTutor, un assistant pédagogique pour des élèves apprenant
8   ↪ Python.
9
10 Tu as accès à 4 outils :
11 - search_python_docs : pour récupérer du contenu de la doc Python officielle
12 - run_python_code    : pour exécuter du code et observer ce qui se passe
13 - generate_exercise  : pour générer un exercice et ses questions
```



```

13 - check_pep8          : pour vérifier le style PEP 8
14
15 Règles de comportement :
16 1. Pour toute question factuelle sur Python (syntaxe, builtin, vocabulaire),
17    utilise search_python_docs AVANT de répondre. Cite la source dans ta réponse.
18 2. Quand l'élève montre un code qui ne marche pas, exécute-le toi-même avec
19    run_python_code AVANT d'expliquer pourquoi, pour voir la vraie erreur.
20 3. Quand tu donnes un exemple de code dans ta réponse, exécute-le d'abord pour
21    être sûr qu'il marche.
22 4. Sois pédagogique : explique POURQUOI, pas seulement COMMENT.
23 5. Adapte ton vocabulaire au niveau de l'élève (déduis-le de ses questions).
24 6. Réponds en français.
25 """
26
27
28 def build_agent():
29     """Construit l'agent PyTutor avec ses 4 tools."""
30     return create_agent(
31         model=get_model(),
32         tools=TOOLS,
33         system_prompt=SYSTEM_PROMPT,
34     )

```

2.1.3 Explications pas à pas

Étape 1 — La signature de create_agent

```

1 create_agent(
2     model,                # instance ChatModel ou string "provider:model"
3     tools,                # liste de @tool callables
4     system_prompt=None,   # str, remplace l'ancien `prompt`
5     # ... d'autres options avancées
6 )

```

Trois paramètres essentiels :

- **model** : une instance issue de `get_model()` (ou n'importe quel `BaseChatModel`). Ou une string courte "anthropic:claude-3-5-sonnet-20241022".
- **tools** : la liste de fonctions décorées `@tool`. L'ordre n'a pas d'importance fonctionnelle, mais peut influencer marginalement les décisions du modèle (priorité au premier tool listé en cas d'ambiguïté).
- **system_prompt** : instructions générales. C'est le *contrat global* de l'agent — son rôle, ses règles, son ton.

Étape 2 — Anatomie d'un system_prompt efficace

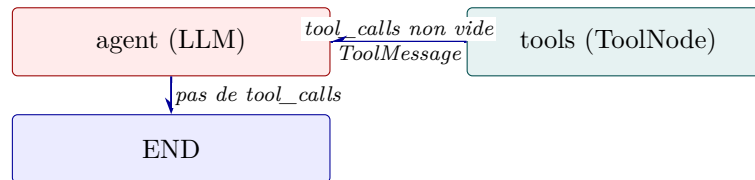
Le SYSTEM_PROMPT de PyTutor a quatre sections distinctes :

1. **Identité** (« Tu es PyTutor, un assistant pédagogique ... »).
2. **Inventaire des tools** (la même liste qu'aux yeux du modèle, en clair). Redondant mais utile : ancre la décision.
3. **Règles d'usage** numérotées qui couvrent les cas critiques (utilise X avant Y, exécute le code avant de l'expliquer, etc.).
4. **Méta-comportement** (langue, ton, public).

*Le `system_prompt` n'est pas un texte décoratif : c'est le **principal levier de qualité** d'un agent. Une règle bien formulée évite des centaines d'appels de tool inutiles. Investis-y du temps.*

Étape 3 — Le graphe LangGraph sous le capot

`create_agent` retourne un **graphe LangGraph compilé**, pas un simple `Runnable`. Visuellement, ce graphe a la forme :



À chaque tour, le nœud LLM examine la conversation. Soit il décide d'appeler des tools (le graphe passe au `ToolNode`, exécute les appels, revient au LLM), soit il produit une réponse finale (le graphe se termine à `END`).

Étape 4 — Invocation : `{"messages": [...]}`

L'agent s'utilise avec un dict contenant la clé `messages` :

```
1 from langchain_core.messages import HumanMessage
2
3 agent = build_agent()
4 result = agent.invoke({
5     "messages": [HumanMessage(content="Comment fonctionne yield ?")]
6 })
7
8 # result["messages"] contient TOUT l'historique :
9 # [SystemMessage, HumanMessage, AIMessage(tool_calls=...), ToolMessage, AIMessage]
```

Le `system_prompt` est injecté automatiquement au début par `create_agent` : tu n'as pas à le passer toi-même dans `messages`.

Étape 5 — Le retour est l'historique complet

`result["messages"]` contient **toute la conversation**, pas seulement la dernière réponse :

- Le `SystemMessage` (ajouté par `create_agent`).
- Le `HumanMessage` initial.
- Les `AIMessage` intermédiaires, parfois avec `tool_calls`.
- Les `ToolMessage` qui contiennent les résultats des tools.
- Le `AIMessage` final (la réponse à l'utilisateur).

C'est cette structure qu'on va lire dans les démos de l'exercice 3.

2.2 Vérification dans LangSmith

L'arbre d'exécution attendu

```
LangGraph (agent compilé)
  agent (LLM) ← tour 1
  tools (ToolNode) ← exécute le(s) appel(s)
  agent (LLM) ← tour 2
  (éventuellement tools → agent à nouveau)
END
```

Trois choses à regarder dans une trace d'agent :

1. Le **system_prompt effectif** dans le premier appel LLM (onglet *Input*). Vérifie qu'il contient tes règles ET les descriptions des tools.
2. Les **tool_calls** dans chaque AIMessage : nom du tool appelé, arguments choisis par le LLM.
3. Les **ToolMessage** : ce que les tools ont *vraiment* renvoyé au LLM. Une réponse incorrecte vient souvent d'un tool qui a renvoyé un truc inattendu.

Récapitulatif

Ce qu'il faut retenir :

1. *`create_agent(model, tools, system_prompt)` construit un agent ReAct prêt à l'emploi en LangChain 1.x.*
2. *Le retour est un **graphe LangGraph compilé** : tout LangGraph s'applique (state, persistance, streaming, etc.).*
3. *Le **system_prompt** doit avoir 4 parties : identité, inventaire des tools, règles numérotées, méta-comportement.*
4. *Invocation : `agent.invoke({"messages": [HumanMessage(...)]})`.*
5. *Le retour est l'historique **complet** de la conversation (System / Human / AI / Tool / AI / ...).*

3 Observer la boucle Réfléchir-Agir-Observer (ReAct)

Exercice 3. Trois démos contrastées pour voir la boucle ReAct

Lance l'agent sur trois requêtes qui déclenchent des comportements différents :

1. **Question factuelle** : doit déclencher `search_python_docs`.
2. **Code qui boucle** : doit déclencher `run_python_code` (l'agent voit le timeout en clair).
3. **Question multi-étape** : doit déclencher *plusieurs* tools en chaîne (recherche + exercice).

Écris une fonction `print_trace(messages)` qui formate l'historique de la conversation pour qu'on *voie* le déroulé de la boucle ReAct.

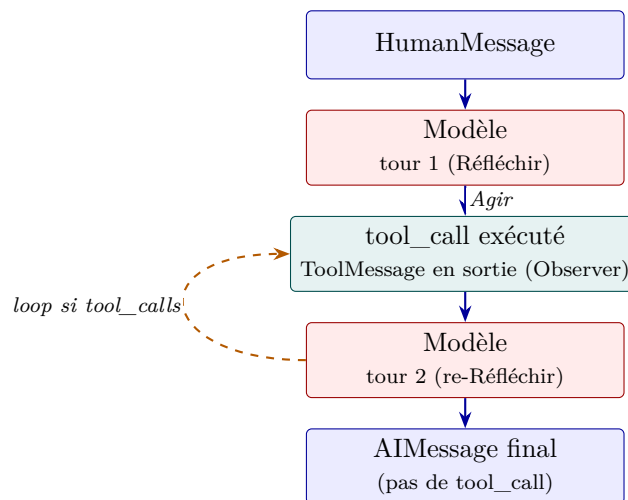
Contrainte : distinguer visuellement `HumanMessage`, `AIMessage` (texte vs `tool_calls`), et `ToolMessage` (résultat des tools).

Objectifs pédagogiques

- Lire l'historique de messages d'un agent : ordre, types, contenu.
- Comprendre la boucle **Réfléchir-Agir-Observer** : le LLM décide → le tool s'exécute → le LLM observe → il re-décide jusqu'à pouvoir répondre.
- Identifier quand un agent décide de *s'arrêter* : l'`AIMessage` sans `tool_calls` est la réponse finale.
- Construire un affichage lisible pour le déroulé pédagogique.

3.1 Solution

3.1.1 Architecture



À chaque tour :

1. Le LLM *réfléchit* : examine la conversation et décide.
2. S'il décide d'appeler un ou plusieurs tools, il les liste dans `AIMessage.tool_calls`.
3. Le `ToolNode agit` : exécute chaque appel et produit un `ToolMessage` par tool.
4. Le LLM *observe* : ces `ToolMessage` entrent dans son contexte au tour suivant.
5. Boucle jusqu'à ce que le LLM produise un `AIMessage` sans `tool_calls` — c'est la réponse finale.

3.1.2 Le code complet

```
1 from langchain_core.messages import (  
2     HumanMessage, AIMessage, ToolMessage, SystemMessage,
```

```

3 )
4
5
6 def print_trace(messages, from_index: int = 0) -> None:
7     """Affiche le déroulé de la boucle ReAct depuis un index donné."""
8     for msg in messages[from_index:]:
9         if isinstance(msg, SystemMessage):
10             continue # on n'affiche pas le system prompt
11         elif isinstance(msg, HumanMessage):
12             print(f"\n[USER]:\n {msg.content}")
13         elif isinstance(msg, AIMessage):
14             if msg.content:
15                 print(f"\n[AI]:\n {msg.content}")
16             if hasattr(msg, "tool_calls") and msg.tool_calls:
17                 print(f"\n[AI] décide d'appeler {len(msg.tool_calls)} tool(s) :")
18                 for tc in msg.tool_calls:
19                     args_preview = str(tc["args"])[0:100]
20                     suffix = "... " if len(str(tc["args"])) > 100 else ""
21                     print(f"    -> {tc['name']}({args_preview}{suffix})")
22             elif isinstance(msg, ToolMessage):
23                 preview = msg.content[:300].replace("\n", "\n    ")
24                 ellipsis = "... " if len(msg.content) > 300 else ""
25                 print(f"\n[TOOL {msg.name}]:\n    {preview}{ellipsis}")
26
27
28 def demo_factual_question() -> None:
29     """Question factuelle : devrait déclencher search_python_docs."""
30     agent = build_agent()
31     result = agent.invoke({
32         "messages": [HumanMessage(content="À quoi sert exactement le mot-clé `yield` ?")]
33     })
34     print_trace(result["messages"])
35
36
37 def demo_buggy_code() -> None:
38     """Code qui boucle à l'infini : devrait déclencher run_python_code."""
39     buggy = """\
40 def double_each(numbers):
41     for n in numbers:
42         numbers.append(n * 2)
43     return numbers
44
45 print(double_each([1, 2, 3]))
46 """
47     agent = build_agent()
48     result = agent.invoke({
49         "messages": [HumanMessage(content=(
50             "Mon code ne se termine jamais. Tu peux m'aider ?\n\n"
51             f"```python\n{buggy}```"
52         ))]
53     })
54     print_trace(result["messages"])
55
56
57 def demo_multi_tool() -> None:
58     """Question complexe : devrait chaîner plusieurs tools."""
59     agent = build_agent()
60     result = agent.invoke({
61         "messages": [HumanMessage(content=(
62             "Explique-moi les list comprehensions en t'appuyant sur la doc, "

```

```

63         "puis génère-moi un exercice de niveau intermédiaire pour pratiquer."
64     )))
65 }
66 print_trace(result["messages"])

```

3.1.3 Explications pas à pas

Étape 1 — La structure de l'historique de messages

`result["messages"]` est une liste hétérogène, dans l'ordre chronologique :

Type	Rôle
SystemMessage	Le <code>system_prompt</code> injecté par <code>create_agent</code> . Toujours en première position.
HumanMessage	Une requête utilisateur.
AIMessage	Une réponse (partielle ou finale) du LLM. Peut contenir : <code>content</code> (texte), <code>tool_calls</code> (liste d'appels à exécuter), ou les deux.
ToolMessage	Le résultat d'un appel de tool. Contient <code>name</code> (nom du tool), <code>content</code> (résultat str), <code>tool_call_id</code> (corrélation avec l' <code>AIMessage.tool_calls</code>).

Étape 2 — La boucle ReAct en pratique

Pour la démo *question factuelle*, on observe typiquement :

1. HumanMessage : « À quoi sert yield ? »
2. AIMessage (tour 1, **Réfléchir**) : pas de `content`, juste `tool_calls = [search_python_docs(query="yield")]`
3. ToolMessage (**Agir + Observer**) : contient les chunks de la doc sur yield.
4. AIMessage (tour 2, **re-Réfléchir**) : `content` = réponse finale, pas de `tool_calls`.

C'est une boucle ReAct à **une seule itération** : l'agent réfléchit, agit une fois, observe, puis répond.

Pour la démo *multi-tool*, la boucle a typiquement 2 itérations : l'agent appelle d'abord `search_python_docs` pour la doc, *observe* le résultat, puis appelle `generate_exercise` pour l'exercice, observe à nouveau, puis répond.

Étape 3 — La condition d'arrêt

L'agent s'arrête **exactement** quand le LLM produit un `AIMessage` sans `tool_calls`. Ce n'est pas un nombre maximal de tours imposé par défaut : c'est le LLM qui décide qu'il a assez d'informations pour répondre.

***Garde-fou de sécurité** : par défaut, le graphe a un `recursion_limit` (typiquement 25 tours) qui force l'arrêt si l'agent boucle sans converger. C'est rare en pratique mais utile pour détecter un bug de prompt (« utilise toujours `search_python_docs` » qui crée une boucle infinie de recherches).*

Étape 4 — `print_trace` : afficher proprement

Trois choix d'affichage :

- Distinguer **visuellement** chaque type de message avec un préfixe (`[USER]`, `[AI]`, `[TOOL nom]`). En production, on utiliserait des emojis ou des couleurs.

- **Tronquer** les contenus longs (`[:300]`) pour garder un déroulé lisible. Sans troncature, un `ToolMessage` avec 4000 caractères de doc rend la trace illisible.
- **Indenter** les contenus pour qu'on voit la structure d'un coup d'œil.

3.2 Vérification dans LangSmith

LangSmith trace chaque agent run avec une visualisation arborescente :

```

LangGraph run
agent (LLM, tour 1)
  Input : [System, Human]
  Output : tool_calls = [{search_python_docs}]
    tools (ToolNode)
      search_python_docs(query="yield")
        Output : "Source: ..."
      agent (LLM, tour 2)
        Input : [System, Human, AI, Tool]
        Output : "Le mot-clé yield..."

```

La vue timeline dans LangSmith montre la durée de chaque tour : on voit instantanément si un tool est lent (souvent `search_python_docs` avec `embedding` + recherche), si le LLM est lent (gros prompt), ou si la boucle a trop d'itérations (probablement un bug de prompt).

Récapitulatif

Ce qu'il faut retenir :

1. L'historique d'un agent est une liste hétérogène : `System`, `Human`, `AIMessage` (avec ou sans `tool_calls`), `ToolMessage`.
2. Boucle ReAct : **Réfléchir** (LLM décide) → **Agir** (tools exécutent) → **Observer** (LLM relit) → re-Réfléchir, jusqu'à un `AIMessage` sans `tool_calls`.
3. L'agent décide seul de s'arrêter : il n'y a pas de nombre fixe de tours, juste un garde-fou `recursion_limit` en cas de bug.
4. Pour debugger un agent : **lit l'historique**, regarde les `tool_calls` et les `ToolMessage`. La plupart des problèmes sont là.

4 Mémoire conversationnelle multi-tours

Exercice 4. Un chat où l'agent se souvient des tours précédents

Construis une boucle de chat (*REPL*) où l'utilisateur peut poser plusieurs questions successives, et où l'agent **se souvient** des tours précédents (utile pour les « peux-tu approfondir ? », « et avec un exemple ? »...).

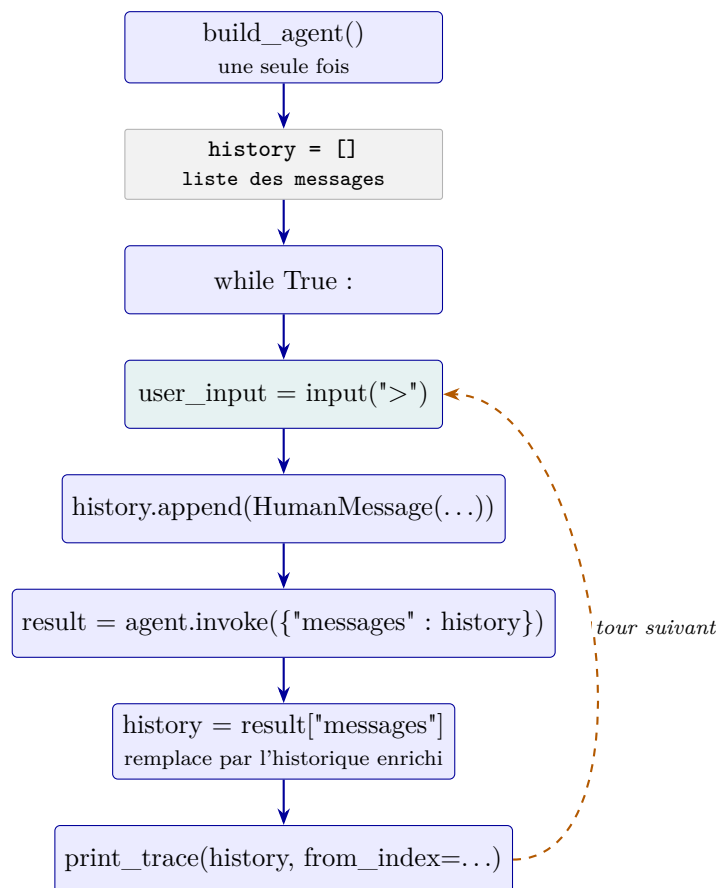
Contrainte : `create_agent` ne stocke *rien* entre les invocations. La mémoire est à maintenir côté client en accumulant l'historique de `messages` et en le réinjectant à chaque tour.

Objectifs pédagogiques

- Comprendre que l'agent par défaut est **stateless** : chaque `invoke` est indépendant.
- Construire la mémoire côté client en accumulant la liste de messages.
- Quantifier le coût (en tokens) d'une mémoire qui s'allonge à chaque tour.
- Connaître les alternatives (`checkpoint` LangGraph) pour une persistance native côté serveur.

4.1 Solution

4.1.1 Architecture



Important : l'agent est construit une seule fois (hors boucle). À chaque tour, on accumule un nouveau `HumanMessage`, on `invoke` avec *tout* l'historique, et on remplace l'historique par celui que renvoie l'agent (qui contient maintenant les nouveaux `AIMessage` et `ToolMessage`).

4.1.2 Le code complet

```
1 def chat_loop() -> None:
2     """Boucle de chat interactive avec mémoire conversationnelle.
3
4     create_agent ne stocke RIEN entre les invocations. Pour avoir une
5     conversation continue, on accumule les messages côté client et on les
6     renvoie à chaque tour.
7     """
8     print("MODE CHAT - Ctrl+C ou ligne vide pour quitter")
9
10    agent = build_agent()
11    history: list = []    # accumulera HumanMessage / AIMessage / ToolMessage
12
13    while True:
14        try:
15            user_input = input("\n> ").strip()
16        except (EOFError, KeyboardInterrupt):
17            print("\nÀ bientôt.")
18            return
19
20        if not user_input:
21            print("Au revoir.")
22            return
23
24        # 1. On ajoute le nouveau message au contexte complet
25        history.append(HumanMessage(content=user_input))
26        index_before = len(history) - 1
27
28        # 2. L'agent renvoie l'historique complet enrichi
29        result = agent.invoke({"messages": history})
30        history = result["messages"]
31
32        # 3. On n'affiche que les messages produits ce tour-ci
33        print_trace(history, from_index=index_before)
```

4.1.3 Explications pas à pas

Étape 1 — Pourquoi l'agent est stateless par défaut

`create_agent` retourne un graphe `LangGraph` compilé qui, par défaut, n'a **pas de checkpoint**. Chaque `.invoke()` exécute le graphe *from scratch* avec uniquement les messages qu'on lui donne en entrée. Aucune mémoire entre appels.

C'est un choix de design : la persistance est un *problème séparé* (où stocker, comment indexer, comment expirer). `LangChain` ne veut pas imposer un backend particulier.

Étape 2 — Maintenir l'historique côté client

Le pattern est simple : on garde une liste `history`, on y ajoute le nouveau message utilisateur, on `invoke`, et on remplace `history` par ce que l'agent renvoie :

```
1 history.append(HumanMessage(content=user_input))
2 result = agent.invoke({"messages": history})
3 history = result["messages"]
```

Après `invoke`, `history` contient :

- Tout l'historique précédent.
- Le nouveau `HumanMessage`.
- Les éventuels `AIMessage(tool_calls=...)` intermédiaires.
- Les éventuels `ToolMessage` avec leurs résultats.
- Le `AIMessage` final.

Au tour suivant, ces messages sont *tous* renvoyés à l'agent — le LLM peut donc se référer aux échanges précédents.

Étape 3 — Le coût en tokens augmente à chaque tour

Tour	Tokens d'entrée du LLM	Cumul
1	system + question 1 (~300 tokens)	300
2	+ AI 1 + Human 2 (~800 tokens)	1100
3	+ AI 2 + Human 3 (~1500 tokens)	2600
...	...	...
10	cumul ~15000 tokens	15000

À chaque tour, on *paie* l'intégralité de l'historique précédent en tokens d'entrée. Sur des conversations longues, ça devient coûteux.

***Mitigation** : tronquer l'historique (garder les N derniers messages), ou résumer les anciens messages avec une chaîne LCEL quand la conversation dépasse une certaine taille (technique de la summary memory).*

Étape 4 — Termination propre

Trois façons de sortir du chat :

- `Ctrl+C` : `input()` lève `KeyboardInterrupt`.
- `Ctrl+D` : `input()` lève `EOFError`.
- **Ligne vide** : `input()` renvoie "", on traite explicitement.

Étape 5 — Alternative serveur : le checkpointer LangGraph

Pour une persistance native (utile dans un backend web), LangGraph fournit des *checkpointers* :

```

1 from langgraph.checkpoint.memory import MemorySaver
2
3 agent = create_agent(
4     model=get_model(),
5     tools=TOOLS,
6     system_prompt=SYSTEM_PROMPT,
7     checkpointer=MemorySaver(), # ou SqliteSaver, PostgresSaver, ...
8 )
9
10 # Chaque conversation est identifiée par un thread_id :
11 config = {"configurable": {"thread_id": "user-42"}}
12 result = agent.invoke(
13     {"messages": [HumanMessage(content="Salut")]},
14     config=config,
15 )

```

Le checkpointer stocke l'état du graphe (donc l'historique) entre les appels. L'utilisateur n'a plus à maintenir `history` côté client.

*Pattern de production : **MemorySaver** pour les tests, un **Sqlite-** ou **Postgres-saver** pour la prod. Avec un **thread_id** par session utilisateur, on a un chat multi-utilisateur sans aucune logique de mémoire côté client.*

4.2 Vérification dans LangSmith

Dans LangSmith, chaque **invoke** produit **une trace distincte** (sauf si on utilise un **trace** explicite). Les traces sont liées par leur projet et leur timestamp, mais pas automatiquement groupées par conversation.

Pour grouper les runs d'un même utilisateur :

- Avec **checkpointer** + **thread_id** : LangSmith expose le **thread_id** dans les métadonnées de chaque run, on peut filtrer.
- Sans **checkpointer** : passer manuellement un **run_name** ou des **tags** pour grouper.

*Astuce : dans la version **stateless** (notre exercice), tu peux comparer le nombre de tokens d'entrée entre les runs successifs pour visualiser la croissance de l'historique. Sur une conversation de 10 tours, c'est édifiant.*

Récapitulatif

Ce qu'il faut retenir :

1. **create_agent** est **stateless** par défaut : chaque **.invoke()** est indépendant.
2. Pour une mémoire conversationnelle simple : accumuler les **messages** côté client, et réinjecter toute l'historique à chaque tour.
3. Le coût en tokens d'entrée **croît à chaque tour**. Sur conversations longues, tronquer ou résumer.
4. Pour une persistance native côté serveur : passer un **checkpointer** à **create_agent** et identifier chaque session par un **thread_id**.
5. Pattern **REPL** standard : init coûteux hors boucle, traitement rapide dans la boucle, terminaison propre sur **Ctrl+C** / **EOF** / ligne vide.